

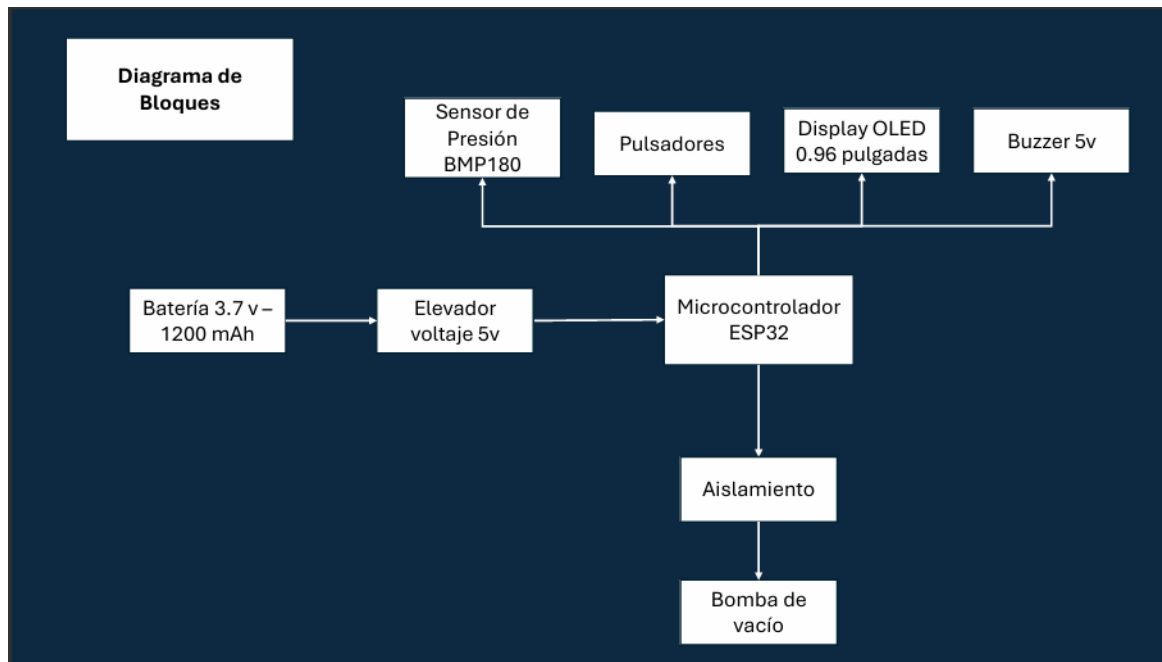
EMBEDDED FIRMWARE DESIGN DOCUMENT

VAC Therapy System — ESP32

Proyecto	proyecto_SE — Terapia de Presión Negativa
Versión FW	2.1.0
Microcontrolador	ESP32 (ESP-IDF + FreeRTOS)
Technical Lead	Matias Berrio
Firmware Engineer	Simon Duarte
HW Integration	Juan Jose Loor

1. Diagrama de Bloques del Sistema Embebido

El sistema está construido alrededor de un microcontrolador ESP32 que integra procesamiento, comunicación inalámbrica y control de actuadores en un único chip. La figura siguiente muestra la arquitectura hardware del sistema.



Esquemático:

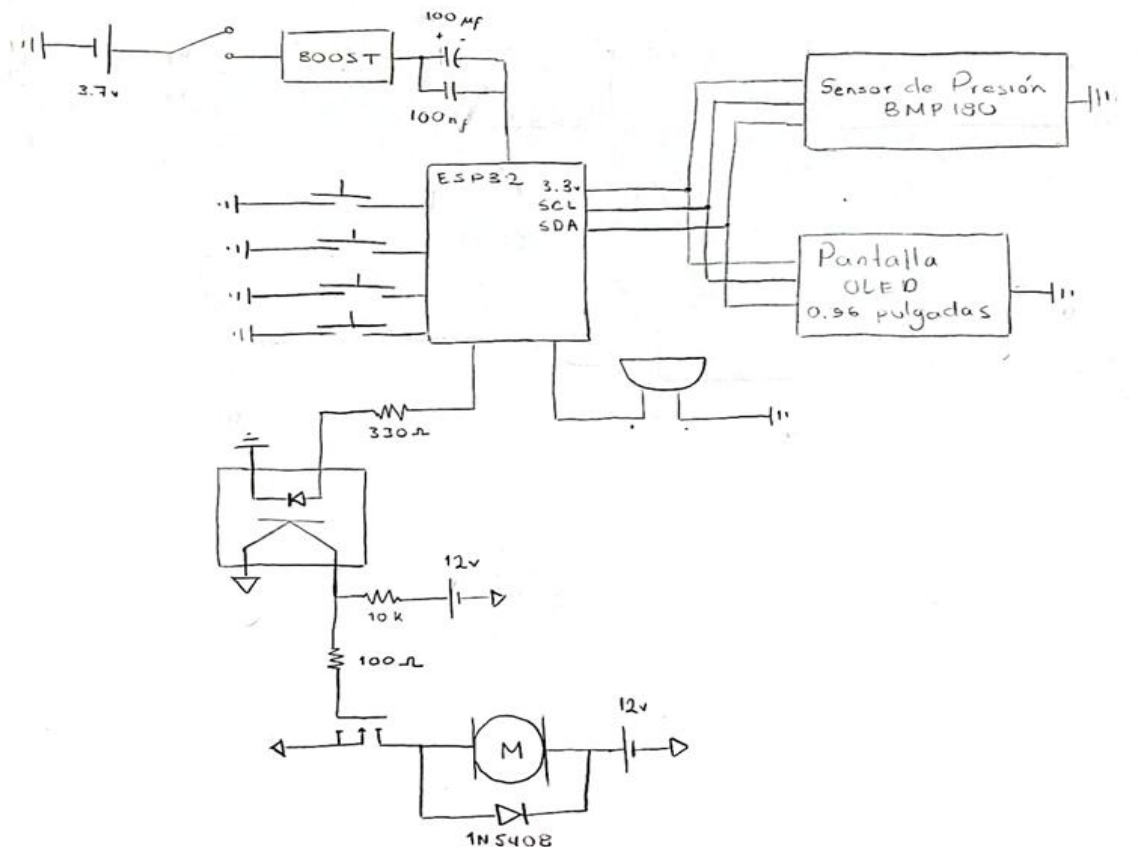


Tabla de interfaces de hardware:

Componente	Protocolo	GPIO / Addr	Dirección	Función
BMP180	I2C	SDA21, SCL22/ 0x77	IN	Sensor de presión
SSD1306 OLED	I2C	SDA21, SCL22/ 0x3C	OUT	Pantalla 128×64
Bomba vacío	GPIO	GPIO 18	OUT	Actuador principal
Buzzer	GPIO	GPIO 19	OUT	Alarma de errores
Botones (×4)	GPIO	GPIO 25,26,27,33	IN	UP/DOWN/ENTER/BACK
App BLE	BLE NUS (NimBLE)	2.4 GHz	IN/OUT	Control remoto / datos

1.1 Justificación de selección de hardware

- ESP32: elegido por su integración de WiFi/BLE en chip, soporte oficial de ESP-IDF + FreeRTOS, y amplia disponibilidad de periféricos (I2C, LEDC/PWM, GPIO). Cumple los requisitos de RF-03 sin hardware externo adicional.
- BMP180: sensor barométrico I2C con resolución suficiente para detectar diferencias de presión en el rango -80 a -125 mmHg. La compensación de temperatura interna mejora la precisión en entornos con variación térmica.
- MOSFET + optoacoplador para bomba: permite control PWM desde GPIO de 3.3V para cargas de 12V, aislando el ESP32 del circuito de potencia.
- Protocolo I2C bus compartido: BMP180 y SSD1306 comparten el bus I2C con direcciones distintas (0x77 y 0x3C), reduciendo el número de pines requeridos.

2. Arquitectura de Firmware

2.1 Modelo de capas

El firmware sigue un modelo de arquitectura estructurado en capas bien definidas que separa estrictamente la interacción con el hardware, la lógica de control y la gestión de la aplicación. Esta separación multidimensional garantiza un acoplamiento mínimo, permitiendo modificar los drivers de los periféricos físicos o el sistema operativo sin alterar el algoritmo de terapia ni la máquina de estados, y viceversa.

Capa	Módulo / Archivo	Responsabilidad
Application	main.c	Inicialización de hardware y módulos. Creación de tareas FreeRTOS
Configuration	app_config.h	Centraliza todos los parámetros: pines, thresholds, periodos de tareas
Middleware	state_machine.c, logger.c, time_manager.c	Lógica de estados del sistema (FSM) y utilidades del sistema de forma agnóstica al hardware.
Control/ Drivers de Sistema	sensor.c, control.c, ble_handler.c, display.c.	Abstracción y validación sobre drivers. Algoritmo de histéresis, lógica de alarmas (fugas, timeouts), menús de interfaz e interpretación de comandos BLE.
HAL	BMP_180.c, SSD1306_MINI.c, ble_nus.c	Comunicación directa mediante protocolos con los chips periféricos (Registros, direcciones I2C).

2.2 Arquitectura de tareas FreeRTOS

El sistema utiliza cuatro tareas FreeRTOS con prioridades diferenciadas. Las tareas de mayor prioridad corresponden a funciones de seguridad (lectura de sensor y control de actuadores), mientras que las de menor prioridad corresponden a la interfaz de usuario y la comunicación BLE no crítica.

Tarea	Prioridad	Período (ms)	Función principal
sensor_task	8 (alta)	100	Lee BMP180, valida lectura, actualiza g_pressure_mmhg
control_task	7	100	Controla bomba y detecta fugas.
ble_task	5	500	Envía DATA: periódico con presión, target y estado
ui_task	4 (baja)	100	Lee botones, actualiza OLED, ejecuta calibración bloqueante
NimBLE host	Interna	Evento	Loop NimBLE para BLE. Llama ble_on_rx() al recibir comando

2.3 Variables compartidas entre tareas

El intercambio de datos clave entre tareas se realiza de forma directa aprovechando el hardware del microcontrolador:

- `g_pressure_mmhg`: Es actualizada por `sensor_task` y consultada por `control_task`, `ui_task` y `ble_task`.
- `g_target_mmhg`: Es modificada por `ui_task` (botones) y `ble_handler` (comandos inalámbricos).

Al tratarse de datos de 32 bits (float e int), la arquitectura Xtensa del ESP32 ejecuta sus lecturas y escrituras de forma atómica (en un solo ciclo de instrucción). Como ninguna tarea depende de una secuencia compleja de lectura-modificación-escritura, se elimina por completo el uso de mutexes, reduciendo la sobrecarga del sistema operativo y evitando bloqueos.

2.4 Estructura de directorios del proyecto

```
proyecto_SE/  
├── main/  
│   ├── app_config.h           ← Parámetros globales (pines, thresholds)  
│   ├── main.c                 ← Init + creación de tareas FreeRTOS  
│   └── CMakeLists.txt  
├── components/  
│   ├── BMP_180/               ← Driver I2C del sensor  
│   │   ├── BMP_180.h / BMP_180.c  
│   │   └── ble_nus/           ← Implementación NimBLE NUS  
│   │       ├── ble_nus.h / ble_nus.c  
│   │       ├── SSD1306_MINI/  ← Driver OLED I2C  
│   │       │   ├── SSD1306_MINI.h / SSD1306_MINI.c  
│   │       ├── state_machine.c/h ← SM global: IDLE/RUNNING/CALIB/ERROR  
│   │       ├── sensor.c/h     ← Validación sobre BMP180  
│   │       ├── control.c/h    ← Control bomba + buzzer + leak detection  
│   │       ├── display.c/h    ← UI OLED + pantalla de alarma  
│   │       ├── logger.c/h     ← Buffer circular de eventos + timestamps  
│   │       ├── time_manager.c/h ← Timestamps con sincronización BLE  
│   │       └── ble_handler.c/h ← Parser de comandos BLE  
└── docs/  
    └── EFDD.docx              ← Este documento
```

2.5 Flujo de trabajo del repositorio

El repositorio sigue un modelo de ramas simple adecuado para equipos de 3 personas:

- `main`: rama principal protegida. Solo acepta merges desde `develop` via Pull Request revisado por el Technical

- Lead.develop: rama de integración continua. Cada tarea de firmware se desarrolla aquí y se integra al final del sprint.
 - Hardware: rama de diseño con esquemáticos, diagramas de bloque y listas de compras. se integran al main cuando el diseño pasa la revisión interna.
-

3. Justificación de Arquitectura

3.1 Separación de responsabilidades

La decisión de separar el firmware en módulos independientes (sensor.c, control.c, display.c, logger.c, state_machine.c, ble_handler.c) responde a principios de ingeniería de software embebido profesional:

- **Mantenibilidad:** un bug en el logger no afecta al control, y viceversa. Cada módulo tiene una única razón para cambiar.
- **Testabilidad:** cada módulo puede probarse con entradas sintéticas sin requerir hardware real (mock de `bmp180_get_pressure_hpa()` para pruebas de `sensor_read()`).

3.2 Uso de FreeRTOS con tareas en lugar de superloop

El firmware original usaba un superloop (`while(1)` en `app_main`). La arquitectura final usa tareas FreeRTOS por las siguientes razones técnicas:

- **Determinismo temporal:** `sensor_task` con prioridad 8 corre cada 100ms independientemente de cuánto tarde la UI en actualizar el OLED. En un superloop, el bloqueo de `bmp180_calibrate()` (64 muestras $\times$ 50ms = 3.2s) detendría todo el sistema.
- **Seguridad:** la tarea de sensor tiene mayor prioridad que la UI. Si la UI se bloquea, el sensor sigue leyendo y el control sigue actuando.
- **Concurrencia limpia:** la calibración es una operación bloqueante. Al ejecutarse en `ui_task` (la de menor prioridad), no bloquea el sensor ni el control durante el proceso.

3.3 app_config.h como punto único de configuración

Centralizar todos los parámetros (`THERAPY_MAX_MMHG`, `LEAK_TIMEOUT_MS`, `SENSOR_TIMEOUT_MS`, etc.) en un único header satisface el RNF-04 y evita magic numbers distribuidos por el código. Cualquier ajuste de umbral requiere modificar un solo archivo, reduciendo el riesgo de inconsistencias.

3.4 Uso de NimBLE en lugar de Bluedroid

NimBLE es la implementación BLE oficial ligera de ESP-IDF. Consume significativamente menos RAM (aprox. 20KB menos) que Bluedroid, lo cual es relevante dado que el ESP32 tiene 520KB de SRAM y el firmware ya utiliza múltiples stacks de tareas FreeRTOS. NimBLE soporta completamente el Nordic UART Service (NUS) con características `WRITE` y `NOTIFY` necesarias para el protocolo del sistema.

3.5 Justificación de Control PWM proporcional ante Fugas Constantes

El sistema opera en presencia de una fuga constante y conocida en el circuito neumático, inherente al diseño del apósito de la terapia VAC. Esta condición hace que un control ON/OFF convencional sea inadecuado: apagaría y encendería la bomba a plena potencia de forma repetitiva, generando oscilaciones de presión y fatiga mecánica prematura. Para esta aplicación se seleccionó un control PWM proporcional puro, justificado bajo los siguientes criterios técnicos:

- **Compensación continua del caudal de fuga:** La bomba permanece activa en todo momento durante la terapia. Cuando el error de presión es grande (la presión está lejos del setpoint), el duty cycle sube proporcionalmente según la relación $\text{duty} = \text{PWM_KP} \times \text{error}$, llevando la bomba a mayor potencia para recuperar el vacío rápidamente. Cuando el error es pequeño o

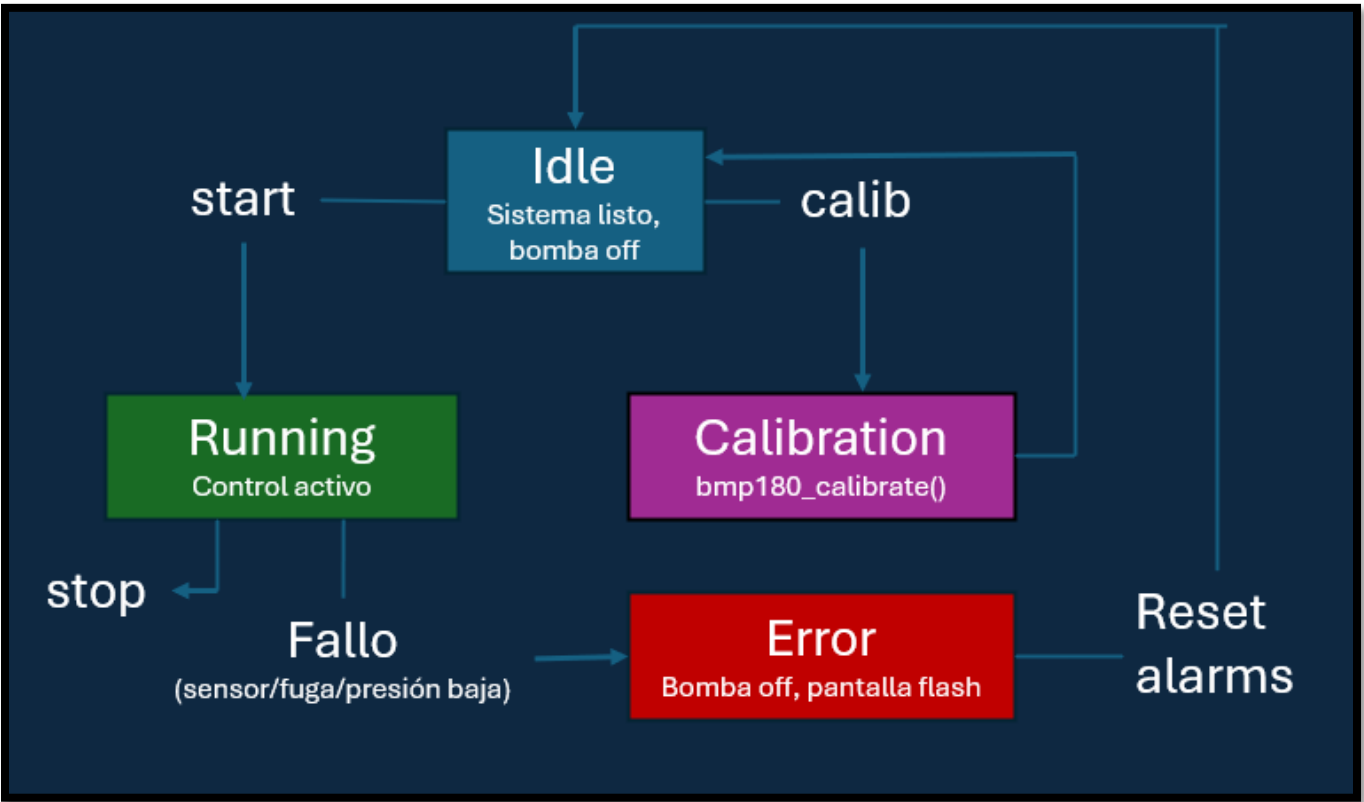
nulo (vacío cerca del objetivo), el firmware aplica un duty cycle mínimo configurable (PUMP_DUTY_MIN_PCT, por defecto 30%) que mantiene un flujo de vacío base continuo. Este mínimo es el mecanismo directo de compensación del leak permanente: evita que la bomba se apague en la zona cercana al setpoint y garantiza que el caudal de fuga sea contrarrestado de forma ininterrumpida.

- **Respuesta proporcional al error:** A diferencia de un controlador ON/OFF que opera siempre a máxima potencia, el duty cycle se ajusta dinámicamente en cada ciclo de ejecución (100 ms) según la magnitud del error instantáneo. Esto produce una respuesta suave y gradual: cuando el sistema está lejos del target la bomba opera a alta potencia, y a medida que se aproxima al setpoint reduce la potencia hasta estabilizarse en el mínimo. El resultado es una presión más estable y predecible que con control todo-o-nada.
- **Protección contra arranques bruscos mediante slew rate:** El firmware limita la velocidad de cambio del duty cycle a un máximo de PUMP_DUTY_SLEW_PCT por ciclo de control (5% cada 100 ms, equivalente a 50% por segundo). Esto evita que al iniciar la terapia o ante un cambio súbito de setpoint la bomba reciba un salto instantáneo de potencia, protegiendo tanto el motor como el MOSFET de picos de corriente. El arranque se realiza de forma gradual independientemente de la magnitud del error inicial
- **Delimitación estricta de seguridad:** Los límites terapéuticos se verifican de forma explícita e independiente del controlador PWM. Si la presión cae por debajo de THERAPY_MIN_MMHG (-125 mmHg), la función control_update() llama inmediatamente a emergency_stop() y declara ERR_PRESSURE_LOW, forzando el sistema a STATE_ERROR con la bomba apagada y el buzzer activado. Este mecanismo opera antes del cálculo de duty y no puede ser inhibido por el controlador proporcional, garantizando que ninguna condición de sobre-vacío pase desapercibida independientemente del estado del sistema.
- **Adaptación implícita a variaciones del entorno:** Al recalcular el duty en cada ciclo según el error actual, el sistema se adapta automáticamente a cambios en la tasa de fuga o en el volumen del circuito neumático sin requerir reconfiguración. Si la fuga aumenta, el error crece y el duty sube proporcionalmente para compensarla. Si la fuga disminuye, el duty reduce hasta el mínimo configurado. No se requiere intervención manual ante cambios en el apósito o en el recipiente de recolección.

4. Diagrama de Estados del Firmware

4.1 Estados del sistema

- IDLE: Estado inicial y de reposo. La bomba está apagada, y el sistema espera un comando START o CALIB. Es el único estado desde el que se puede iniciar la terapia o calibración.
- RUNNING: Terapia activa. control_task ejecuta el algoritmo de PWM que actúa sobre la bomba, y monitorea fugas. sensor_task actualiza continuamente la presión.
- CALIBRATION: Proceso bloqueante de 64 muestras (~3.2 segundos). Se ejecuta en ui_task para no bloquear sensor ni control. La bomba permanece apagada durante la calibración.
- ERROR: Estado de falla. emergency_stop() fue llamado y la bomba está apagada. El sistema permanece en este estado hasta que el operador ejecuta RESET_ALARMS via BLE.



4.2 Tabla de transiciones

Estado Origen	Estado Destino	Condición / Evento	Acción ejecutada
IDLE	RUNNING	START (BLE/botón)	control_reset_leak_timer(), sensor_reset_faults(), log 'Terapia iniciada'
IDLE	CALIBRATION	CALIB (BLE/botón DOWN)	display → SCREEN_CALIBRATION, bmp180_calibrate() en ui_task
RUNNING	IDLE	STOP (BLE/botón)	Bomba OFF, log 'Terapia detenida'

RUNNING	ERROR	sensor_read() false × THRESHOLD	emergency_stop(), sm_set_error(ERR_SENSOR_*), log error
RUNNING	ERROR	Fuga: bomba ON > LEAK_TIMEOUT sin target	emergency_stop(), sm_set_error(ERR_LEAK_DETECTED)
RUNNING	ERROR	pressure < THERAPY_MIN_MMHG	emergency_stop(), sm_set_error(ERR_PRESSURE_LOW).
CALIBRATION	IDLE	bmp180_calibrate() completado	sensor_set_offset(), ble 'OK:CALIB_DONE', log calibración
ERROR	IDLE	RESET_ALARMS (BLE) con sm_reset_error()	s_error = ERR_NONE, sensor_reset_faults(), display → MENU

4.3 Comportamiento de emergency_stop()

La función emergency_stop() es el punto único de entrada a la condición de falla activa. Su implementación garantiza:

- Bomba apagada: gpio_set_level(PIN_PUMP, 0) — acción directa sobre GPIO, no depende del scheduler.
- Reset del leak timer: s_target_reached = false, s_pump_start_tick = 0.
- El log se registra antes de retornar, garantizando trazabilidad del evento.

5. Estrategia de Manejo de Errores

5.1 Clasificación de errores

El sistema distingue dos niveles de error, ambos gestionados a través de `state_machine.c` mediante `sm_set_error()`:

- Errores de sensor (`ERR_SENSOR_TIMEOUT`, `ERR_SENSOR_RANGE`): originados en `sensor.c` tras `SENSOR_FAULT_THRESHOLD` lecturas inválidas consecutivas. Requieren desconexión física del BMP180 o fallo eléctrico sostenido para dispararse.
- Errores operativos (`ERR_LEAK_DETECTED`, `ERR_PRESSURE_LOW`): originados en `control.c` durante la ejecución del algoritmo de control. Representan condiciones de seguridad del paciente.

Código de Error	Condición de disparo	Módulo origen	Acción del sistema
<code>ERR_SENSOR_TIMEOUT</code>	Sin lectura válida por <code>>SENSOR_TIMEOUT_MS</code> (2000ms)	<code>sensor.c</code>	<code>emergency_stop()</code> , <code>sm_set_error()</code> , pantalla ERROR, BLE notifica
<code>ERR_SENSOR_RANGE</code>	<code>hPa < 300</code> o <code>hPa > 1100</code> , o NaN/Inf en lectura	<code>sensor.c</code>	<code>emergency_stop()</code> , <code>sm_set_error()</code> , log error sensor
<code>ERR_LEAK_DETECTED</code>	Bomba ON <code>>LEAK_TIMEOUT_MS</code> (15s) sin alcanzar target	<code>control.c</code>	<code>emergency_stop()</code> , <code>sm_set_error()</code> , log 'FUGA: bomba sin target'
<code>ERR_PRESSURE_LOW</code>	<code>pressure_mmhg < THERAPY_MIN_MMHG</code> (-125 mmHg)	<code>control.c</code>	Bomba OFF, <code>sm_set_error()</code> , log presión límite

5.2 Mecanismo de debounce y thresholds

Para evitar falsas alarmas por eventos transitorios (glitches I2C, variaciones momentáneas de presión), el sistema implementa debounce por conteo de eventos consecutivos:

Parámetro	Valor y justificación
<code>SENSOR_FAULT_THRESHOLD</code>	5 lecturas inválidas consecutivas. Evita falsas alarmas por glitches I2C transitorios. Una sola lectura mala no dispara el error.
<code>SENSOR_TIMEOUT_MS</code>	2000ms. El BMP180 tarda aprox. 20ms por lectura completa (10ms temp + 10ms presión). 2000ms = 100 ciclos sin lectura es inequívocamente un fallo.
<code>LEAK_TIMEOUT_MS</code>	15000ms. El tiempo típico para alcanzar -100 mmHg desde atmósfera es 2-5s según el volumen del sistema. 15s da margen amplio antes de declarar fuga.
<code>SAFETY_MARGIN_MMHG</code>	Alarma a 10 mmHg del límite de vacío para evitar falsos disparos por el ruido de $\pm 1-2$ mmHg del sensor BMP180 sin comprometer el límite de seguridad (-125 mmHg)

5.3 Flujo de respuesta ante falla

1. `sensor_read()` detecta lectura inválida → incrementa `s_fault_count`.
2. Si `s_fault_count >= SENSOR_FAULT_THRESHOLD`: llama `emergency_stop()` y `sm_set_error(code)`.
3. `emergency_stop()`: `gpio_set_level(PIN_PUMP, 0)`
4. `sm_set_error()`: `state = STATE_ERROR`, `s_error = code`, `logger_add(mensaje)`.
5. `ui_task` detecta `sm_is_error() == true` → `display_set_screen(SCREEN_ALARM)` con parpadeo + `gpio_set_level(PIN_BUZZER, 1)`.
6. `ble_task` envía DATA: con `state=ERROR` en el siguiente ciclo de 500ms.
7. Operador envía RESET_ALARMS via BLE o presiona `BTN_ENTER` → `sm_reset_error()` + `sensor_reset_faults()`.
8. Sistema vuelve a `STATE_IDLE`. El operador debe evaluar la causa antes de reiniciar terapia.

5.4 Garantía fail-safe

La función `emergency_stop()` actúa de forma inmediata mediante control directo sobre los pines físicos (`gpio_set_level`). Como esta función de ESP-IDF es segura entre hilos (*thread-safe*) y opera de forma independiente al planificador (*scheduler*) de FreeRTOS, garantiza el apagado inmediato de la bomba incluso si el sistema operativo está sobrecargado o bloqueado.

6. Estrategia de Logging

6.1 Diseño del logger

El módulo logger.c implementa un buffer circular de `LOGGER_BUFFER_SIZE` entradas (32) en RAM estática. Cada entrada almacena un timestamp y un mensaje de texto de hasta `LOGGER_MSG_LEN` (64) caracteres. No se usa heap dinámico para evitar fragmentación y garantizar comportamiento determinístico.

6.2 Thread-safety

El buffer está protegido por un mutex FreeRTOS (`SemaphoreHandle_t s_mutex`). La función `logger_add()` toma el mutex, escribe la entrada, y lo libera. La función `logger_dump_ble()` libera y retoma el mutex entre cada envío BLE para no bloquear otras tareas durante el dump.

6.3 Comportamiento de buffer lleno

Cuando el buffer alcanza `LOGGER_BUFFER_SIZE` entradas, la siguiente escritura sobrescribe la entrada más antigua. La cabeza de escritura (`s_head`) avanza circularmente. La entrada más antigua siempre es accesible a través de la lógica del índice `start`.

6.4 Timestamps y sincronización

El ESP32 no tiene RTC. `time_manager.c` mantiene dos modos: modo sincronizado (comando `TIME:<epoch>` recibido via BLE) y modo uptime (segundos desde el boot). En modo sincronizado, el timestamp es `epoch_base + (ticks_ahora - ticks_sync) / 1000`, calculado sin división de punto flotante.

Estructura interna del buffer:

```
typedef struct {
    uint32_t timestamp;           // Epoch Unix o uptime [segundos]
    char      msg[64];           // Mensaje del evento (texto libre)
} log_entry_t;
```

Formato de salida BLE (comando GET_LOGS):

```
LOG:0,1716415023,Sistema iniciado v2.1.0
LOG:1,1716415025,Terapia iniciada
LOG:2,1716415087,FUGA: bomba sin alcanzar target
LOG:3,1716415090,Error reseteado por usuario
LOG:END
```

6.5 Eventos registrados

- Sistema iniciado (boot con versión)
- Terapia iniciada / detenida
- Calibración iniciada / completada
- Error de sensor (timeout o fuera de rango)
- Fuga detectada (bomba sin alcanzar target)
- Presión excede límite inferior
- Error reseteado por usuario

7. Gestión de Comunicaciones

7.1 Protocolo I2C — Sensor y Display

El bus I2C es compartido por el sensor BMP180 (dirección 0x77) y la pantalla OLED SSD1306 (dirección 0x3C). La inicialización del bus se realiza una única vez en `i2c_init()` del `main.c`, evitando la doble instalación del driver que ocurría en la versión anterior del firmware.

Parámetro	BMP180	SSD1306	Justificación
Velocidad	100 kHz	100 kHz	Standard Mode (100kHz). Ambos dispositivos soportan Fast Mode (400kHz) pero 100kHz reduce ruido en el bus compartido.
Dirección	0x77	0x3C	Direcciones fijas en hardware. No hay conflicto en el bus compartido.
Pull-ups	Internos ESP32	Internos ESP32	GPIO_PULLUP_ENABLE en ESP-IDF. Resistencias ~47kΩ internas, suficientes para 100kHz con capacitancia de bus baja.
Timeout I2C	100ms	100ms	<code>pdMS_TO_TICKS(100)</code> . Si el dispositivo no responde en 100ms, la función retorna error (NACK), lo cual <code>sensor.c</code> interpreta como lectura inválida.

7.2 Protocolo BLE — Nordic UART Service (NUS)

El sistema implementa el Nordic UART Service (NUS), un protocolo de transporte genérico sobre BLE que emula una UART bidireccional. Se eligió NUS sobre un servicio GATT personalizado por:

- Compatibilidad: NUS es soportado nativamente por aplicaciones de depuración BLE estándar (nRF Toolbox, Serial Bluetooth Terminal, LightBlue) sin configuración adicional.
- Simplicidad: el protocolo es texto plano, facilitando la depuración y el desarrollo de la app de PC.
- El stack NimBLE en ESP-IDF tiene menor huella de memoria que Bluedroid, dejando más RAM disponible para las tareas de control.

Comando BLE (RX)	Respuesta (TX)	Descripción y validación
START	OK:START / ERR:...	Solo válido en STATE_IDLE. Si se rechaza, responde con el estado actual.
STOP	OK:STOP / ERR:...	Solo válido en STATE_RUNNING. Transición a IDLE.
SET:<mmHg>	OK:SET:<val>	Setpoint clamped a [-80, -125]. <code>_clamp_target()</code> asegura

		rango terapéutico. Responde el valor real aplicado.
CALIB	OK:CALIB iniciada / ERR	Solo válido desde STATE_IDLE. ui_task ejecuta bmp180_calibrate() de forma bloqueante en su propia tarea.
STATUS	STATUS:state=...,target=...,pump=...,err=...,ts=...	Respuesta completa del sistema. Incluye estado SM, setpoint, bomba, error activo y timestamp.
GET_STATE	STATE:<estado>	Retorna solo el estado de la SM. Útil para polling rápido desde la app.
GET_ERRORS	ERR_ACTIVE:<code> / NONE	Retorna el error activo actual. ERR_NONE si el sistema está sin falla.
GET_LOGS	LOG:idx,ts,msg (×N) + LOG:END	Dump completo del buffer circular. Pausa de 20ms entre paquetes para no saturar el stack BLE.
RESET_ALARMS	OK:RESET_ALARMS / ERR	Solo válido en STATE_ERROR. Llama sm_reset_error() + sensor_reset_faults(). Sistema vuelve a IDLE.
TIME:<epoch>	OK:TIME / ERR:TIME	Sincroniza timestamp. Epoch válido si > 1000000000 (aprox. año 2001). Actualiza time_manager.
Datos periódicos	DATA:presion,target,pump,state	Enviado automáticamente cada 500ms si el cliente está suscrito a notificaciones (notify_enabled).

7.3 Manejo de comandos malformados

El parser en ble_handler.c usa strncasecmp() para comparación case-insensitive. Cualquier comando no reconocido responde con ERR:CMD_UNKNOWN [<comando>], lo que facilita el diagnóstico desde la app. No se genera un error de sistema (sm_set_error) por comandos inválidos — solo se registra un warning con ESP_LOGW.

8. SRTM — Software Requirements Traceability Matrix

Req ID	Requirement	Test case ID	Test type	Artifacts
RF-01	Control PWM para bomba de vacío	TC-01	HIL	Foto/Video (Bomba operando, captura de osciloscopio del PWM)
RF-02	Autocalibración y medición de presión (BMP180)	TC-02	Integration	Foto/Video, Captura de la GUI (OLED en modo calibración)
RF-03	Detección y tolerancia a fallos del sensor	TC-03	HIL	Foto/Video (Desconexión física), Captura de la GUI (Alarma en OLED)
RF-04	Detección automática de fugas masivas	TC-04	System	Foto/Video (Prueba de fuga), Captura de la GUI (Alarma ERR_LEAK_DETECTED)
RF-05	Protección <i>Fail-Safe</i> contra sobre-vacío	TC-05	System	Foto/Video, Captura de la GUI (Alarma ERR_PRESSURE_LOW)
RF-06	Interfaz visual y despliegue de telemetría	TC-06	System	Foto/Video, Captura de la GUI
RF-07	Registro de historial de eventos (Logger)	TC-07	Integration	Captura de la GUI (App BLE mostrando el historial descargado)
RF-08	Comunicación inalámbrica y telemetría BLE	TC-08	Integration	Captura de la GUI (App móvil recibiendo datos y enviando comandos)
RF-09	Control de estados de la terapia (Inicio / Pausa / Parada)	TC-09	System	Foto/Video, Captura de la GUI (OLED indicando estado de PAUSA)

RF-10	Priorización de alarmas críticas en pantalla	TC-10	System	Foto/Video, Captura de la GUI (Mensaje de alerta intermitente)
RNF-01	Carcasa funcional y protección	TC-11	System	Foto/Video (Ensamble final y pruebas de uso)
RNF-02	Alimentación aislada y protección del microcontrolador	TC-12	HIL	Foto/Video (Mediciones de osciloscopio en el optoacoplador, voltajes de batería)
RNF-03	Configurabilidad en tiempo de ejecución	TC-13	System	Foto/Video, Captura de la GUI (OLED y App BLE sincronizadas)
RNF-04	Ensamble tarjeta universal	TC-14	HIL/System	Foto tarjeta soldada

9. Casos de Prueba

Test Case	TC-01	Estado	
Requerimiento	RF-01: Control PWM para bomba de vacío		
Objetivo	Verificar que la bomba modula su potencia mediante un control proporcional puro en función del error de presión instantáneo, manteniendo un flujo base mínimo ante fugas, limitando los cambios bruscos por <i>slew rate</i> y respetando el corte de seguridad.		
Precondiciones	1. ESP32 y etapa de potencia (MOSFET) alimentados de forma independiente y operativos.		
	2. Sensor BMP180 calibrado y conectado al bus I2C.		
	3. Circuito neumático cerrado con la fuga constante controlada e inherente al apósito activa.		
	4. Parámetros cargados en firmware: PUMP_DUTY_MIN_PCT = 30%, PUMP_DUTY_SLEW_PCT = 5% por ciclo (100 ms) y <i>Setpoint</i> inicial (ej. -100 mmHg).		
Resultados esperados		Resultados Obtenidos	
1. Control Proporcional: El ciclo de trabajo (<i>duty cycle</i>) escala de forma gradual según la magnitud del error actual, evitando oscilaciones bruscas ON/OFF.			
2. Mínimo de Fuga: Al aproximarse al <i>setpoint</i> , la bomba mantiene un piso fijo del 30% PWM para mitigar el leak permanente de aire de forma ininterrumpida.			
3. Slew Rate: Los cambios súbitos de potencia se limitan a un 5% por ciclo (100 ms), suavizando la rampa de corriente en el motor y el MOSFET.			
Evidencias			

Test Case	TC-02	Estado	
Requerimiento	RF-02: Autocalibración y medición de presión (BMP180)		
Objetivo	Validar que el firmware toma lecturas estables a través de I2C y ejecuta la rutina de autocalibración promediando las 64 muestras consecutivas para establecer el cero relativo (presión atmosférica local).		
Precondiciones	1. Sistema en estado inicial (terapia detenida). 2. Circuito neumático expuesto por completo a la presión atmosférica actual (sin vacío retenido). 3. Sensor BMP180 energizado con comunicación I2C activa.		
Resultados esperados		Resultados Obtenidos	
1. Al activar la función de calibración, la interfaz OLED muestra el estado del proceso y bloquea temporalmente el control neumático. 2. El firmware lee y promedia exactamente 64 muestras rápidas. 3. Se almacena con éxito el offset y la pantalla pasa a reportar un valor inicial estable de 0 mmHg.			
Evidencias			

Test Case	TC-03	Estado	
Requerimiento	RF-03: Detección y tolerancia a fallos del sensor		
Objetivo	Comprobar la robustez del sistema y la respuesta de la máquina de estados ante la pérdida imprevista de comunicación con el sensor de presión BMP180.		
Precondicione s	1. Sistema operando en estado activo de terapia (bomba modulando). 2. Acceso físico a los cables de conexión del bus de datos I2C (SDA/SCL).		
Resultados esperados		Resultados Obtenidos	
1. Tras interrumpir las líneas I2C, el firmware detecta la ausencia de respuesta por timeout (2000 ms) o registra 5 muestras erráticas fuera de rango. 2. La máquina de estados corta inmediatamente la señal PWM de la bomba (0% duty). 3. El sistema entra de forma segura en STATE_ERROR y reporta el código correspondiente.			
Evidencias			

Test Case	TC-04	Estado	
Requerimiento	RF-04: Detección automática de fugas masivas		
Objetivo	Verificar que el firmware interrumpe la terapia de manera segura si la tasa de escape neumático supera la capacidad máxima de la bomba.		
Precondicione s	1. Terapia activa y bomba operando de manera normal. 2. Presión estabilizada dentro del rango terapéutico.		
Resultados esperados		Resultados Obtenidos	
1. Al retirar de golpe el apósito o desconectar la manguera principal, la presión cae drásticamente y el algoritmo eleva el PWM progresivamente al 100%. 2. El temporizador de seguridad interna arranca la cuenta y valida la persistencia del fallo. 3. Tras 15 segundos continuos sin llegar a la presión objetivo, la bomba se detiene por completo y se activa el código de error ERR_LEAK_DETECTED.			
Evidencias			

Test Case	TC-05	Estado	
Requerimiento	RF-05: Protección Fail-Safe contra sobre-vacío		
Objetivo	Validar la desconexión de emergencia inmediata en caso de lecturas de presión neumática excesivas que vulneren los límites terapéuticos de seguridad.		
Precondicione s	1. Sistema ejecutando el lazo de control dinámico en terapia activa. 2. Mecanismo externo preparado para forzar un incremento abrupto de vacío en el circuito.		
Resultados esperados		Resultados Obtenidos	
1. Al superar artificialmente el umbral de seguridad de -125 mmHg + margen, el algoritmo de control cancela toda ejecución regular. 2. Se apaga inmediatamente la energía del motor de la bomba. 3. El sistema bloquea el funcionamiento ordinario y activa la pantalla intermitente con el código ERR_PRESSURE_LOW.			
Evidencias			

Test Case	TC-06	Estado			
Requerimiento	RF-06: Interfaz visual y despliegue de telemetría				
Objetivo	Asegurar la actualización constante, legible y precisa de los parámetros de la sesión médica en la pantalla integrada durante condiciones de operación estándar.				
Precondicione s	1. Pantalla OLED inicializada correctamente por el firmware. 2. Terapia en ejecución con variaciones dinámicas en la presión del circuito.				
Resultados esperados		Resultados Obtenidos			
1. El display muestra de forma fija la estructura básica de la interfaz (Etiquetas de estado, Setpoint y Presión actual). 2. Los valores numéricos cambian en tiempo real reflejando de forma idéntica las variables procesadas en la tarea de control. 3. No se aprecian parpadeos molestos, deformación de fuentes o congelamiento de la GUI.					
Evidencias					

Test Case	TC-07	Estado	
Requerimiento	RF-07: Registro de historial de eventos (Logger)		
Objetivo	Comprobar que el firmware encola cronológicamente los últimos 32 eventos del sistema en el búfer circular alojado en la memoria RAM, gestionando el acceso concurrente sin bloqueos ni salidas serie activas.		
Precondicione s	1. El firmware ha inicializado el Mutex de protección del logger. 2. El sistema realiza transiciones ordinarias o simulaciones de errores sucesivos.		
Resultados esperados		Resultados Obtenidos	
1. Al generar eventos de forma consecutiva, el índice de cabeza avanza ciclicamente y sobrescribe las entradas más antiguas si supera el tamaño de 32. 2. Cada log se guarda de forma estructurada con su estampa de tiempo epoch. 3. Al descargar los datos por BLE, el arreglo se vuelca secuencialmente y sin corrupciones.			
Evidencias			

Test Case	TC-08	Estado	
Requerimiento	RF-08: Comunicación inalámbrica y telemetría BLE		
Objetivo	Verificar la correcta vinculación inalámbrica mediante el servicio NUS y evaluar el correcto procesamiento bidireccional de la telemetría y strings de comando ASCII.		
Precondicione s	1. Stack Bluetooth activo en el ESP32 emitiendo tramas de advertising. 2. Aplicación móvil compatible lista en el dispositivo de prueba.		
Resultados esperados		Resultados Obtenidos	
1. El terminal móvil descubre y enlaza con éxito el servicio inalámbrico del sistema. 2. La aplicación recibe de manera asíncrona las cadenas de telemetría continuas. 3. Al transmitir comandos desde el teléfono (ej. START, SET:100), la máquina de estados interpreta el string y ejecuta la acción correspondiente en el equipo.			
Evidencias			

Test Case	TC-09	Estado	
Requerimiento	RF-09: Control de estados de la terapia (Inicio / Pausa / Parada)		
Objetivo	Validar la capacidad de pausar y restablecer las operaciones del circuito neumático bajo demanda, garantizando que no se pierdan las configuraciones establecidas ni los registros previos en memoria.		
Precondicione s	1. Terapia activa, bomba modulando y parámetros cargados en el sistema.		
Resultados esperados		Resultados Obtenidos	
1. Al oprimir el comando de Pausa, la señal PWM se reduce al 0% y los contadores temporales se congelan.			
2. La interfaz cambia a modo "PAUSA" manteniendo visible el setpoint previo.			
3. Al reactivar la terapia, el motor reanuda su modulación suave partiendo de los valores exactos definidos antes de la interrupción.			
Evidencias			

Test Case	TC-10	Estado	
Requerimiento	RF-10: Priorización de alarmas críticas en pantalla		
Objetivo	Comprobar que cualquier pantalla de error de la FSM interrumpe de forma jerárquica y prioritaria las pantallas secundarias de configuración o monitorización de datos.		
Precondicione s	1. Sistema encendido. 2. Operario interactuando en las subpantallas de ajustes o navegación ordinaria.		
Resultados esperados		Resultados Obtenidos	
1. Al gatillar una falla forzada (ej. fuga masiva), el flujo del menú actual es cancelado e invalidado por la tarea gráfica. 2. El display OLED cambia de forma inmediata a un formato de advertencia explícito y de alta prioridad. 3. El mensaje se muestra parpadeante y permanece bloqueado en pantalla sin dejar retornar al usuario a los menús comunes hasta corregir la falla.			
Evidencias			

Test Case	TC-11	Estado	
Requerimiento	RNF-01: Carcasa funcional y protección		
Objetivo	Evaluar la solidez física, el correcto acoplamiento de las piezas mecánicas fabricadas en impresión 3D y la protección general de los componentes internos del sistema.		
Precondicione s	1. Carcasa de plástico completamente impresa en 3D (carcasa principal y tapa). 2. Todo el hardware y la neumática instalados de forma fija en su interior mediante los soportes diseñados.		
Resultados esperados		Resultados Obtenidos	
1. Las piezas de la carcasa encajan perfectamente sin holguras ni deformaciones mecánicas. 2. La pantalla OLED, los botones y los puertos para las mangueras de vacío se alinean correctamente con las aperturas impresas, permitiendo su operación sin obstrucciones. 3. Los circuitos internos quedan aislados y protegidos del exterior ante manipulación ordinaria.			
Evidencias			

Test Case	TC-12	Estado	
Requerimiento	RNF-02: Alimentación aislada y protección del microcontrolador		
Objetivo	Verificar que la circuitería lógica del ESP32 está eléctricamente aislada de la etapa transitoria de potencia de la bomba de 12V a través del acoplamiento optoelectrónico y batería autónoma.		
Precondiciones	1. Circuito ensamblado con la batería conectada al ESP32 y la línea de 12V conectada independientemente a la bomba. 2. Puntas de osciloscopio vinculadas a ambos extremos de la sección aislada del optoacoplador.		
Resultados esperados		Resultados Obtenidos	
1. El ESP32 se enciende y ejecuta tareas usando únicamente su alimentación dedicada. 2. Las mediciones eléctricas demuestran la transferencia de la onda PWM sin vinculación física de tierras o voltajes entre potencia y lógica.			
Evidencias			

Test Case	TC-13	Estado	
Requerimiento	RNF-03: Configurabilidad en tiempo de ejecución		
Objetivo	Confirmar la capacidad de alterar dinámicamente los parámetros operativos esenciales de la terapia médica en caliente, evitando flasheos de código o reinicios del procesador.		
Precondicione s	1. Terapia neumática en funcionamiento constante. 2. Acceso a la botonera o aplicación remota BLE.		
Resultados esperados		Resultados Obtenidos	
1. Al modificar el setpoint por cualquiera de las vías disponibles, el sistema altera de inmediato los valores cargados en las variables en RAM.			
Evidencias			

Test Case	TC-14	Estado	
Requerimiento	RNF-04: Ensamble en tarjeta universal		
Objetivo	Verificar que todos los componentes estén correctamente soldados y distribuidos sobre la tarjeta universal según el diseño esquemático.		
Precondicione s	1. Prototipo electrónico completamente ensamblado y soldado en la tarjeta universal. 2. Continuidad eléctrica verificada previamente en los rieles principales de alimentación y tierra para evitar cortocircuitos.		
Resultados esperados		Resultados Obtenidos	
1. No se presentan componentes sueltos o falsos contactos mecánicos/eléctricos durante la operación de la bomba. 2. Las soldaduras son firmes, limpias y no existen puentes accidentales de estaño entre pistas.			
Evidencias			